The End of Human-Scale AppSec

Rebuilding Security for the AI-Native SDLC

Ken Johnson · CTO & Co-Founder, DryRun Security



DRYRUN SECURITY



ABOUT KEN

I've spent my career building, breaking, and rethinking AppSec.

Ken Johnson · CTO & Co-Founder, DryRun Security

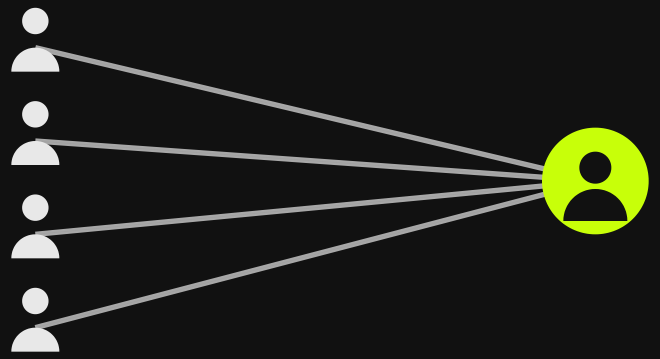
- **19 years** in application security
- Co-host of the Absolute AppSec podcast
- Longtime OWASP contributor; Black Hat and DEF CON trainer
- Led AppSec at GitHub and LivingSocial
- Creator of RailsGoat, WeirdAAL, and WXF
- Now building agentic systems for contextual code security analysis

THE QUESTION

What happens when software stops being produced at human scale?

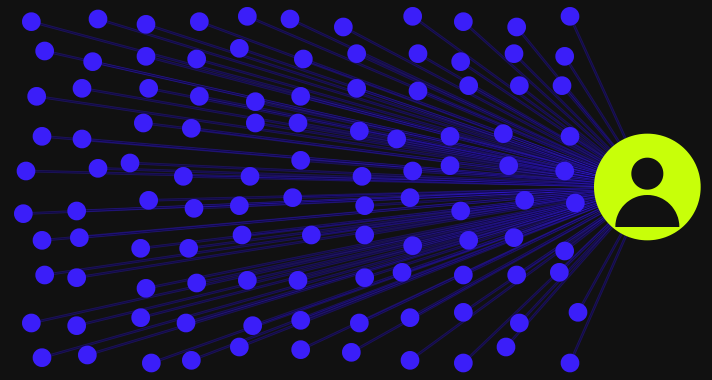
The volume grows. The security program doesn't.

Human scale



A few developers, one security engineer: manageable

Agentic scale



A swarm of agents, the same one security engineer

ACT I

The hidden human architecture of AppSec

Our tools shaped the work

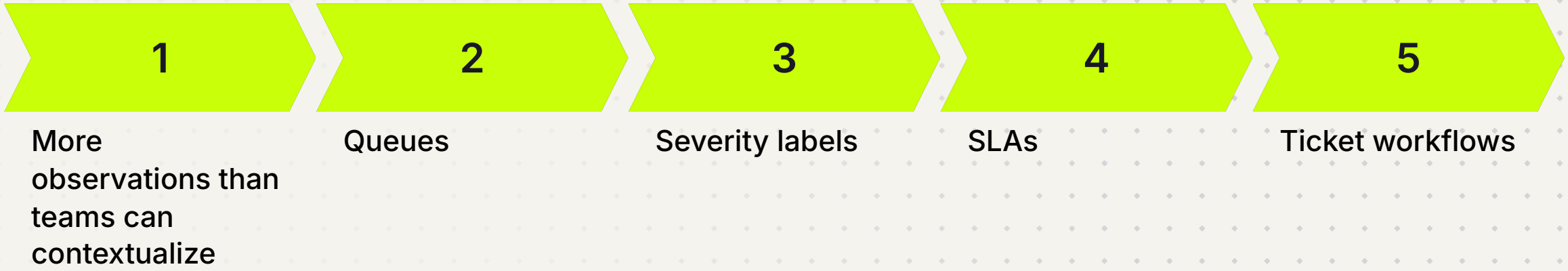
1. Scanner produces observations
2. Human validates
3. Human reconstructs context
4. Human determines impact
5. Human prioritizes
6. Human creates ticket
7. Human negotiates remediation

One step is automated. Six belong to a person.

The practitioner connects everything the tools cannot



The backlog was an adaptation to tool limitations



A rational response, not practitioner failure.

We started managing findings instead of managing risk

WHAT WE MANAGE

A finding backlog

Static rows, counted and aged.

WHAT ACTUALLY MATTERS

A service whose risk keeps changing

New code, new exposure, new dependencies every week.

Findings are evidence about risk, not the risk state.

Finding more vulnerabilities does not prove that risk went down

ACTIVITY

- Findings created
- Tickets opened
- Scans completed
- Mean time to triage

OUTCOMES

- Faster Remediation
- Controls verified
- Exposure reduced
- Unsafe patterns prevented
- High-risk services improved

AppSec exists to reduce risk in software

BUILD IT SAFE

Secure design, threat modeling, guardrails and paved roads, secure frameworks and libraries

CHECK IT

Code and architecture review, security testing, developer guidance

RESPOND AND GOVERN

External report validation, incident support, organizational risk management

ACT II

The human integration layer no longer scales

Two things are changing at the same time

01

AI is changing how software is delivered

More code, more services, more changes, produced faster than people can read them.

+

02

AI can interpret and synthesize data in novel ways

Systems can now do security work our old tools never could.

More code is only half of the story.

The development side is ambitious

AI-native development visions place agents across the entire lifecycle.

01 Planning

02 Design

03 Implementation

04 Testing

05 Review

06 Deployment

07 Maintenance

Every stage gets agents.

Security gets documentation and another review step

01 Update security requirements

02 Update a threat model

03 Run a review tool

04 Produce findings

A broad agentic system narrows into four small steps.

Example: Anthropic published AI-native SDLC playbook.
https://claude.com/resources/articles/the-ai-native-sdlc-playbook

We are not bug janitors

- Design
- Architecture
- Guardrails
- Education

AppSec

`scanning` `review`

`findings` `remediation`

- Incident response
- Consultation
- External reports
- Organizational risk

We are rebuilding yesterday's tools with more AI

SAST

+ AI

DAST

+ AI

SCA

+ AI

Tickets

+ AI

Dashboards

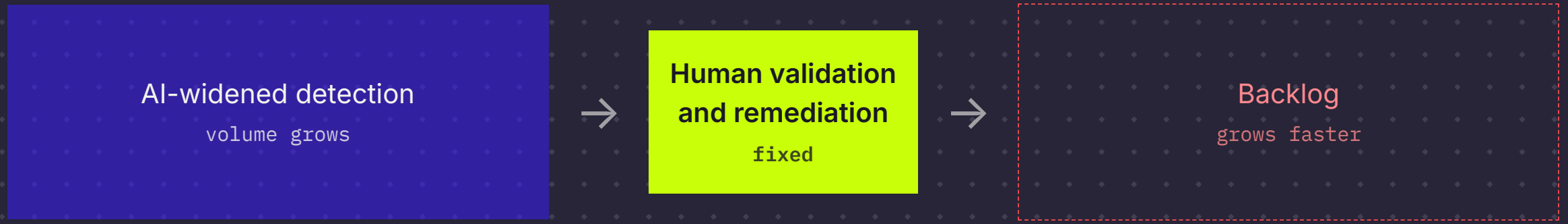
+ AI

Review gates

+ AI

Agents added. The underlying work unchanged.

Faster discovery can create a faster backlog



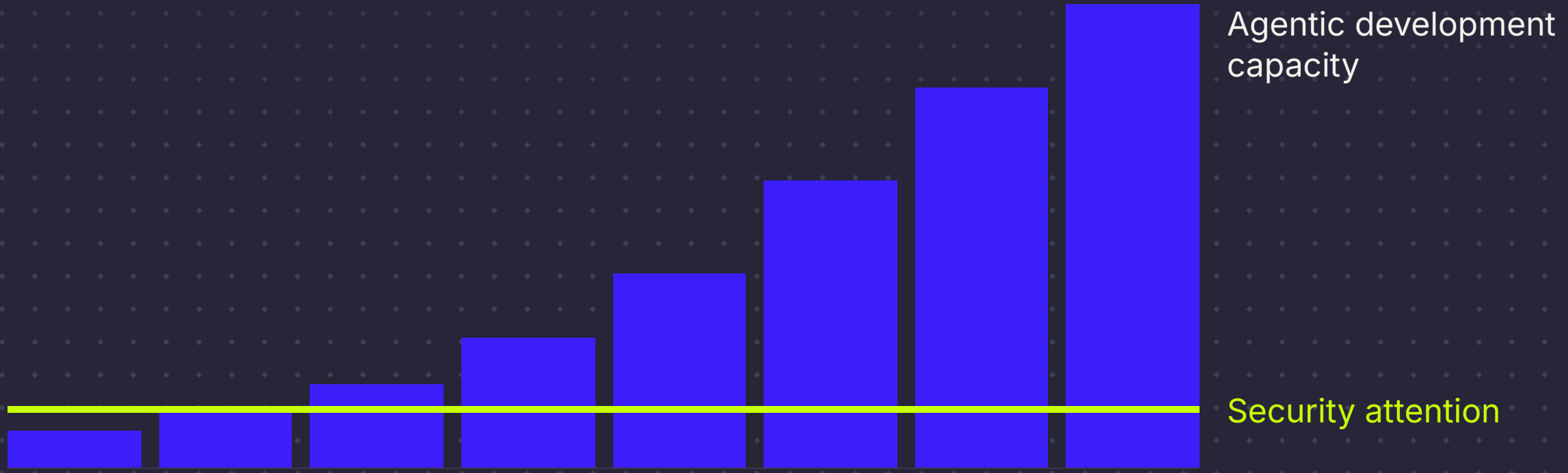
Why "AI-powered D/I/S-AST" can amplify the old failure.

Our processes assume humans are the primary producers

- Humans write most changes, at reviewable velocity
- Security can sample the important work
- A human can reconstruct missing context
- Current processes remain scalable controls

Four assumptions hiding in every AppSec program.

Software production expands. The human control plane does not.



You cannot hire your way through a broken operating model

Doing the work doesn't scale. Governing the work can.

Humans doing the work

- Experts personally run every threat model, design consult, and risk decision
- Capacity grows one hire at a time
- Agentic change outpaces it

Humans governing agentic work

Practitioner governs

- Define security properties
- Build guardrails and paved roads
- Set evidence and escalation rules
- Evaluate the system

Agent workflows

- Gather context
- Maintain knowledge state
- Verify controls
- Track service risk state

Capacity grows with the agents

We can produce a larger backlog, or change the operating model

AI-scale software production

PATH 1 · MORE OF THE SAME

More generation

More scanning

More findings

More human triage

Larger backlog

PATH 2 · CHANGE THE MODEL

Continuous context

Reasoned investigation

Verified controls

↻ Changing application risk state

ACT III

Change the center of the system: continuous risk-state management

From vulnerability backlog to application risk state

OLD CENTER

Finding backlog

→

NEW CENTER

Evolving risk state of the service

Risk is the totality of what we know about the service

SHAPE

- Architecture and trust boundaries
- Exposure and data sensitivity
- Ownership and operational health

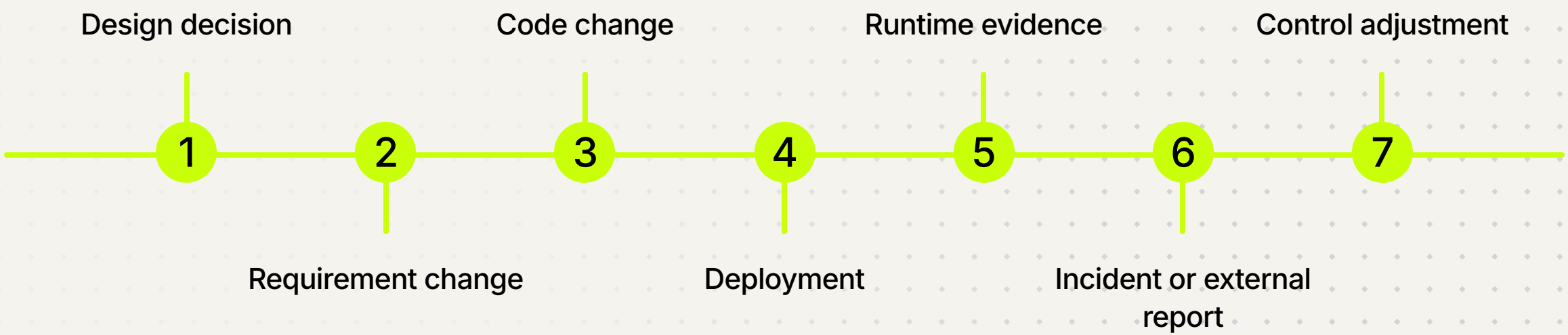
WEAKNESS

- Technology age and dependencies
- Known vulnerabilities and attack paths
- Threat-model coverage

CHANGE

- Control effectiveness
- Significant recent changes
- Incidents and external reports
- Accepted and accumulating risk

Risk state is dynamic



AppSec operates at two connected levels

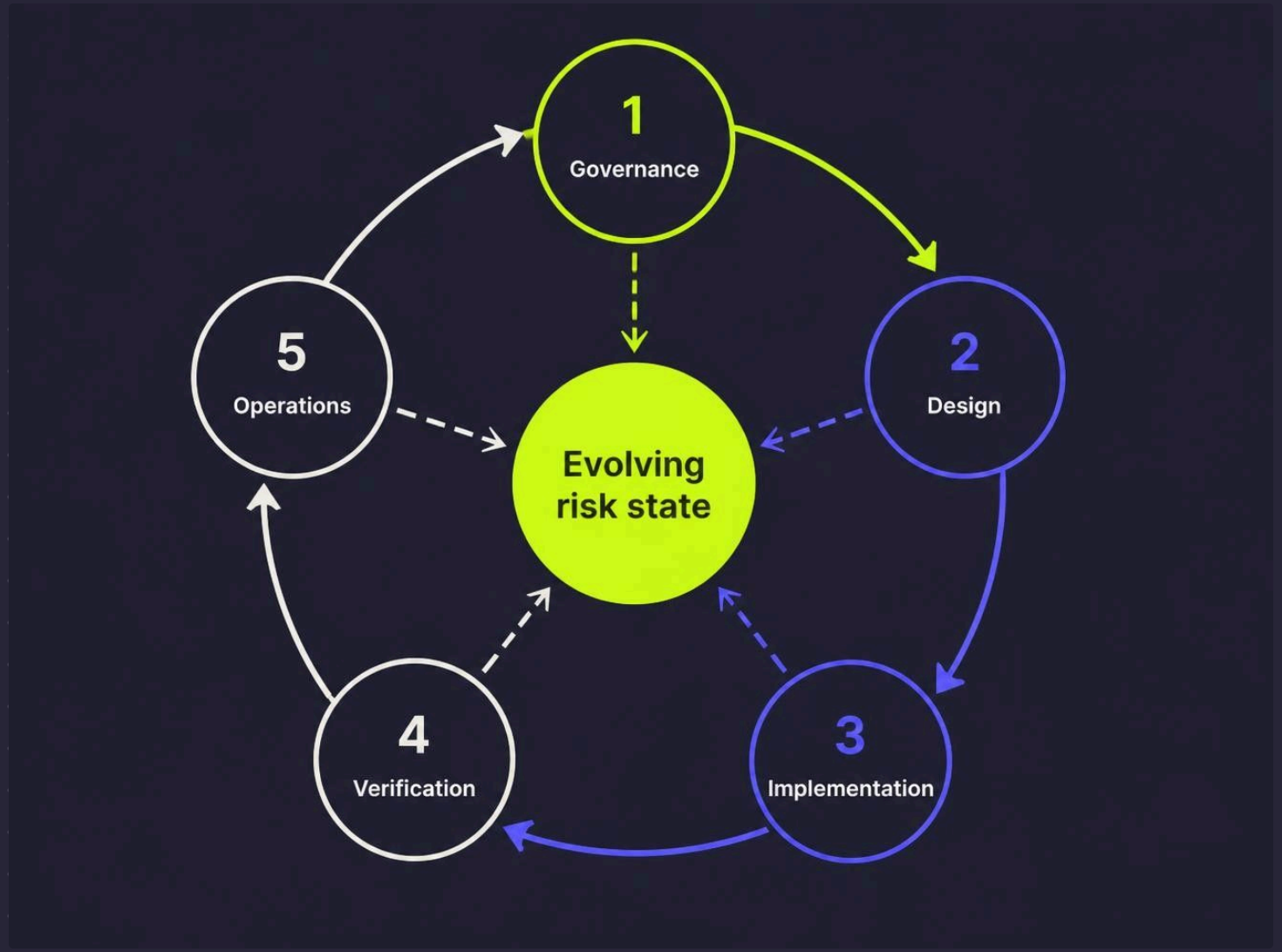
Service level

Maintain the evolving risk state. Evaluate significant changes. Keep threat and architecture context. Connect findings into attacker outcomes. Verify controls. Incorporate runtime evidence and incidents.

Program level

Define strategy and security properties. Build guardrails, secure frameworks, and paved roads. Govern agentic systems. Measure risk across services. Find categorical failures. Allocate expert attention.

AppSec already spans the entire software system



OWASP SAMM's five business functions as a connected loop, not pillars.

SAMM / GOVERNANCE

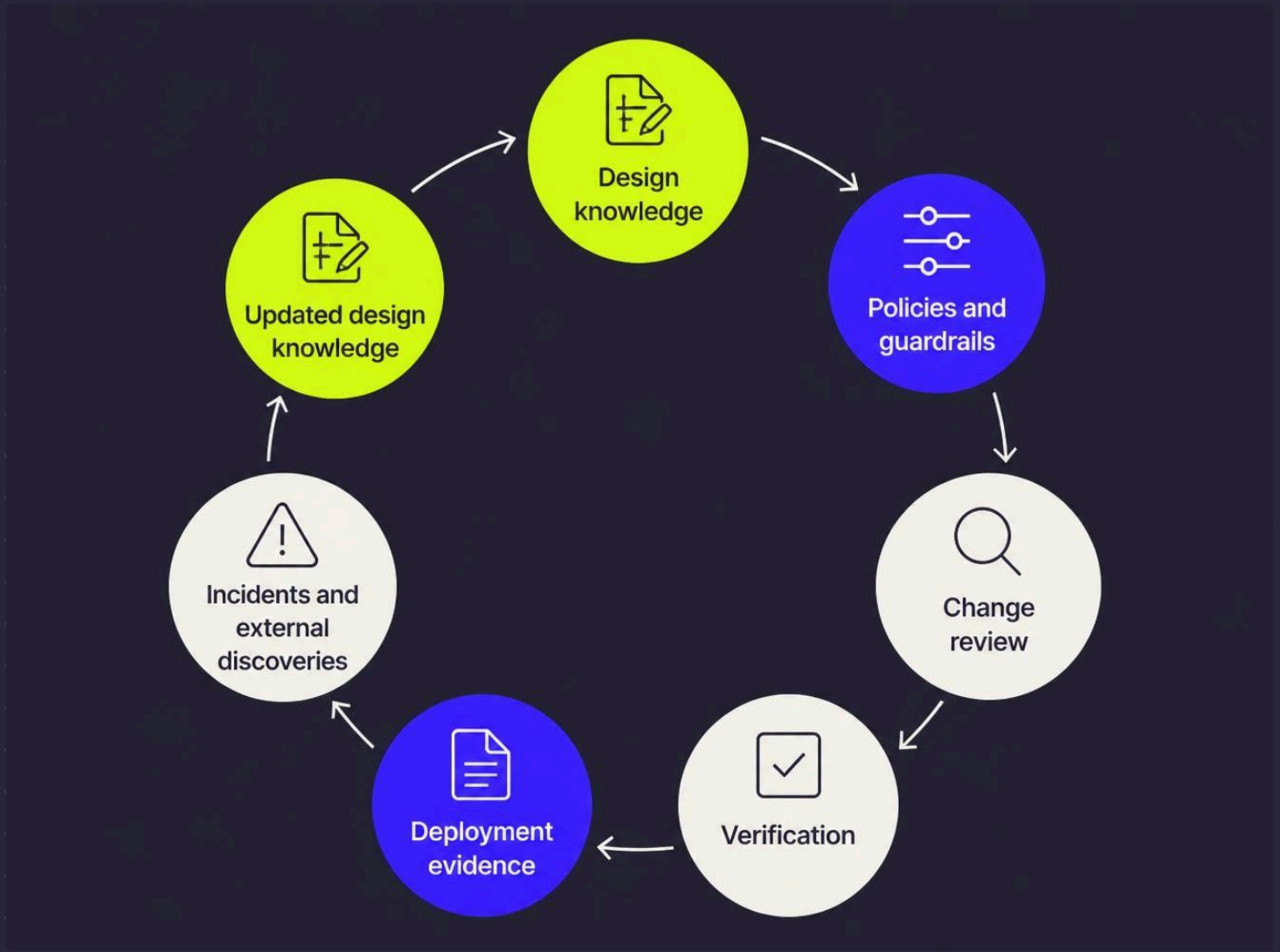
Know what matters, who owns it, and whether controls work

- Service-level risk
- Ownership
- Exposure and business importance
- Risk appetite

- Policies and security properties
- Control effectiveness
- Organizational trends

A continuously updated decision layer.

Security knowledge follows the software



Every stage teaches the next.
Nothing is filed and forgotten.

The old product categories begin to dissolve

SAST

Isolated source findings → agentic investigation across changes, architecture, data flow, and context

DAST

Periodic external probing → hypothesis-driven attack exploration and runtime verification

SCA

Dependency inventory → reachability, exploitability, malicious package behavior, and application impact

Finding management

Ticket repository → evidence and application risk knowledge

Three observations can describe one attacker outcome

OBSERVATION

A client-controlled limit

OBSERVATION

A server that trusts the counter

OBSERVATION

A paid upstream service

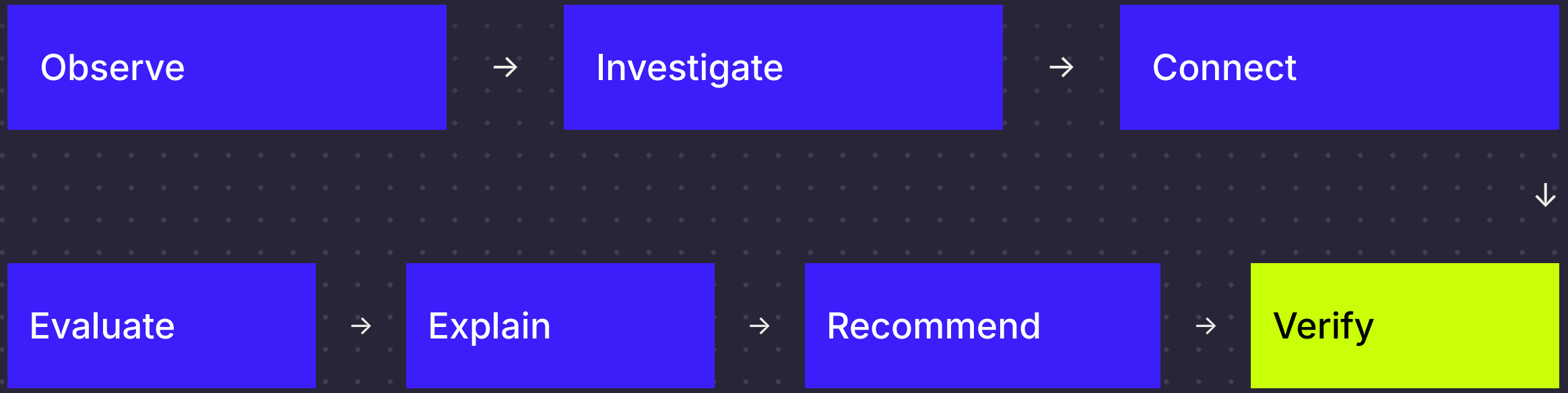


CONCLUSION

Unbounded paid API abuse

A generic example.

Intelligent systems should do more of the security work



The output is a reasoned conclusion with evidence, not a bigger inventory.

Human work moves upward

PRACTITIONERS ↑

Define security properties · challenge architecture · build guardrails · resolve ambiguity · make consequential risk decisions · govern the system

AGENTIC SYSTEMS

Gather context · inspect changes · maintain models · test hypotheses · correlate evidence · verify controls · track risk state

The hackers and the builders move closer together

APPSEC

Adversarial thinking, investigation, exploit reasoning, risk judgment

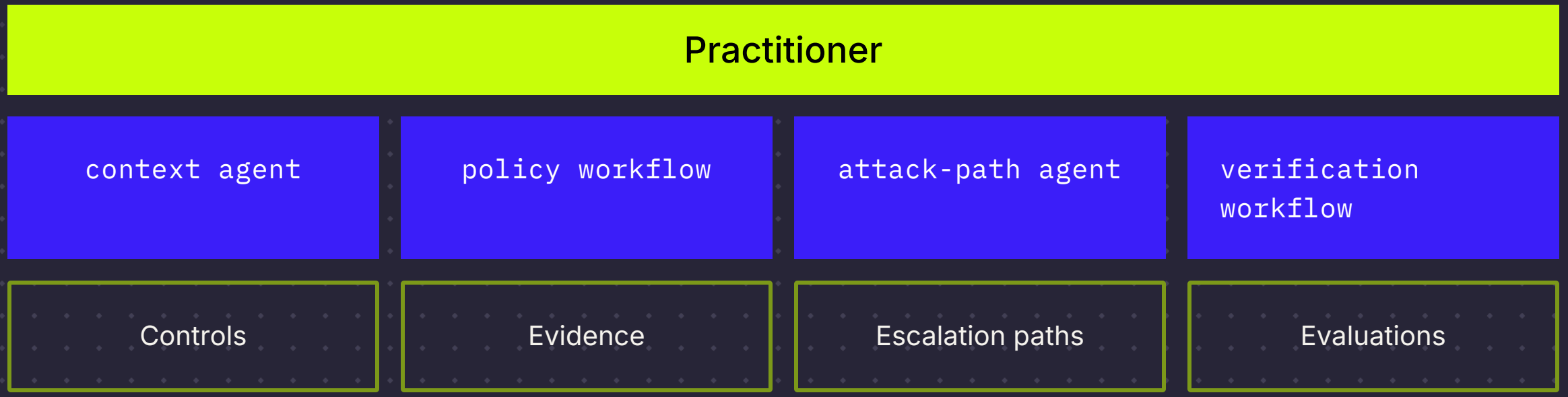
SHARED

Software-literate practitioners who can both challenge and build

PRODUCT SECURITY

Guardrails, secure libraries, platform integration, developer enablement

The future AppSec practitioner governs systems of work



ACT IV

The answer is not blind autonomy

AI depends on what humans have already learned

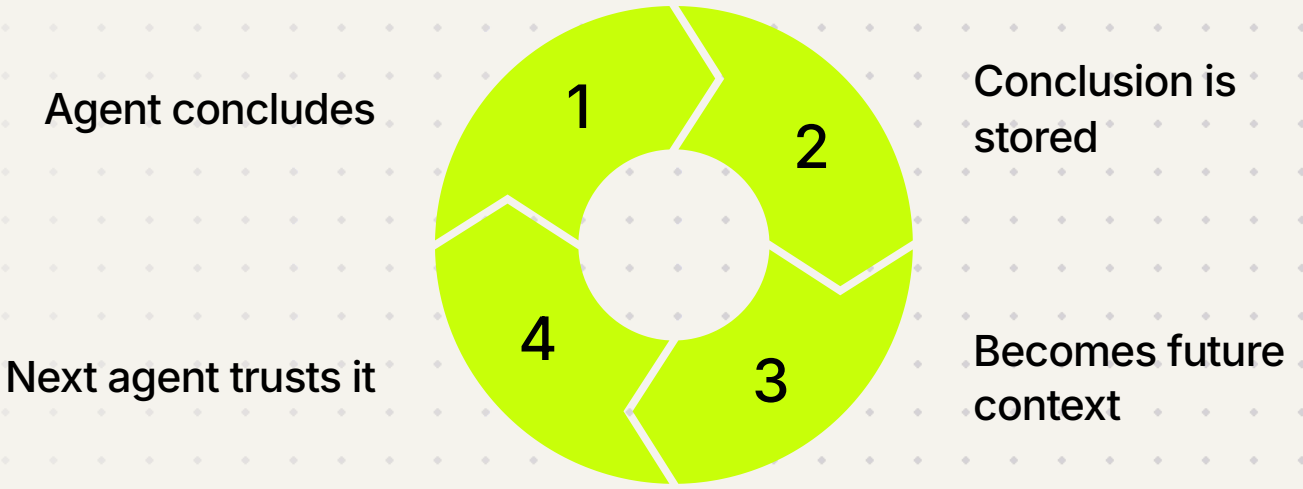
KNOWLEDGE BOUNDARY

- Vulnerability classes
- Exploit techniques
- Secure patterns
- Incident lessons
- Architecture knowledge
- Organizational policies

A novel failure

Human discovery still finds it.

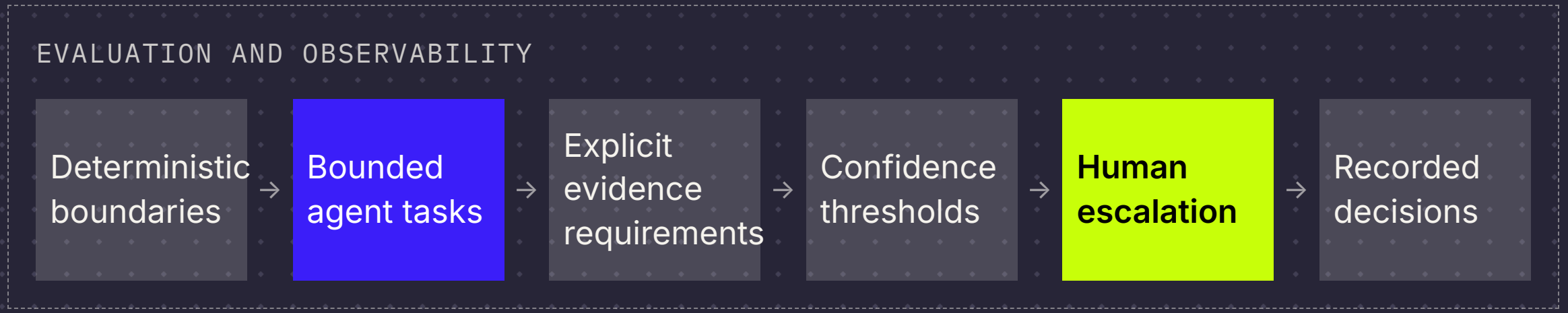
Automated judgment must not become inherited certainty



Generated conclusions become future context. Unverified assumptions compound.

Trust in autonomous security systems has to be continuously earned and continuously verified.

Be disciplined about where uncertainty enters the system



Build bounded security workflows, not one giant security agent

WEAK

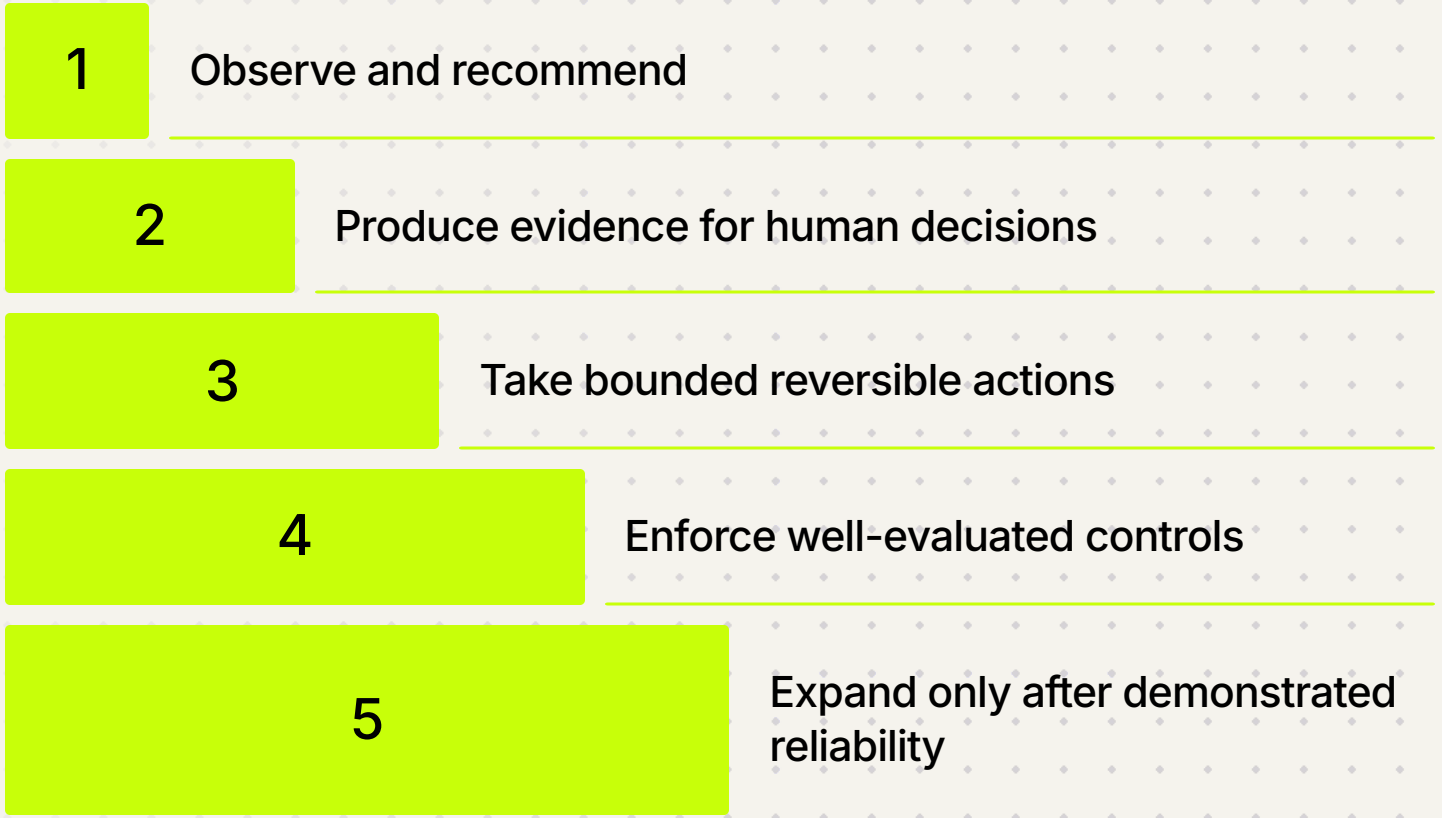
One agent

Broad access, vague goal to "secure the application."

STRONGER

- Context gathering
- Policy evaluation
- Attack-path analysis
- Verification
- Reporting
- Escalation

Autonomy should expand with evidence



Each step is earned, never assumed.

Do not evaluate only the output

1. Input and retrieved context
2. Tool access and permissions
3. Reasoning workflow and decomposition
4. Final conclusion and action

Evaluate the conditions that produced the answer.

Start where expert attention is being wasted

1. Identify repetitive work consuming expert judgment
2. Decompose it into bounded workflows
3. Define required evidence and security properties
4. Set deterministic boundaries and permission limits
5. **Keep humans at consequential decision points**
6. Evaluate reliability, failure modes, and drift
7. Expand autonomy only when evidence supports it

A transition you can begin with the systems you already have.

Measure changes in risk, not agent activity

DON'T OPTIMIZE FOR

Agent runs · findings generated · tickets opened · automated comments · apparent task completion

OPTIMIZE FOR

Material risks removed · unsafe patterns prevented · controls verified · time returned to practitioners · evidence-backed decisions · service-level risk improvement

AppSec becomes the control system for software risk



Around the software organization, not bolted on at the end.



If the SDLC is being rebuilt,

AppSec has to be rebuilt with it.

The End of Human-Scale AppSec · Ken Johnson,
CTO & Co-Founder, DryRun Security

https://linkedin.com/in/cktricky
ken@dryrunsecurity.com